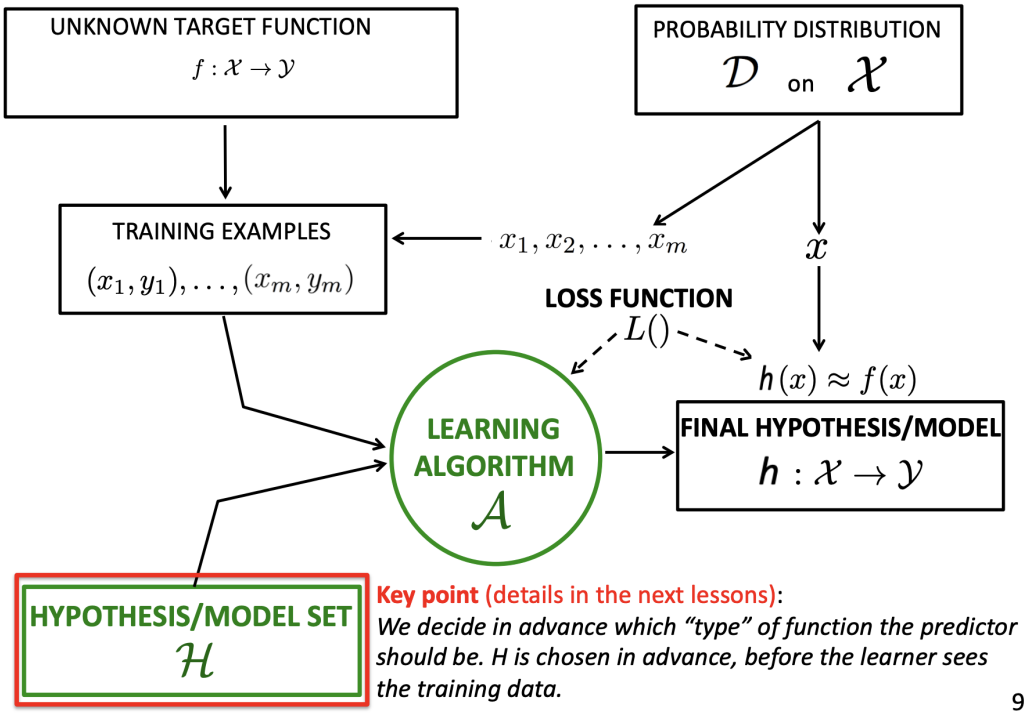
1) Formal Learning Model

The learner, which is the machine, has access to the following:

- **Domain Set** X: set of all possible objects to label
  we call $x \in X$ the instance
- **Label Set** Y: is the set of possible labels
- **Training Data** $S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$: *finite* sequence of labeled $X \times Y$ domain points. Used as *input*
- **Learner's Output, Prediction Rule** $h : X \to Y$: this is the rule produced by the algorithm: we call $A(S)$ the output from the algorithm A when the training set S was applied
- **Data Generation Model** in our cases instances are generated by a distribution D and labeled according to a function f, both D and f are not known to the learner, only the outputs of the elements in S
- **Measure Of Success**: this is the probability to predict the *wrong* label

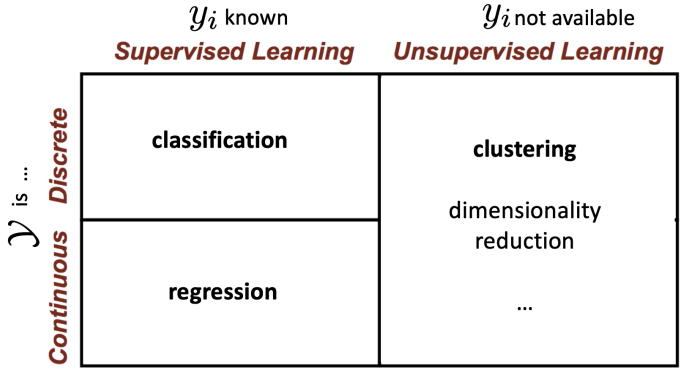
$$L_{D,f} := P_{x \sim D}[h(x) \neq f(x)] := D(\{x : h(x) \neq f(x)\})$$



Model

Types of Learning

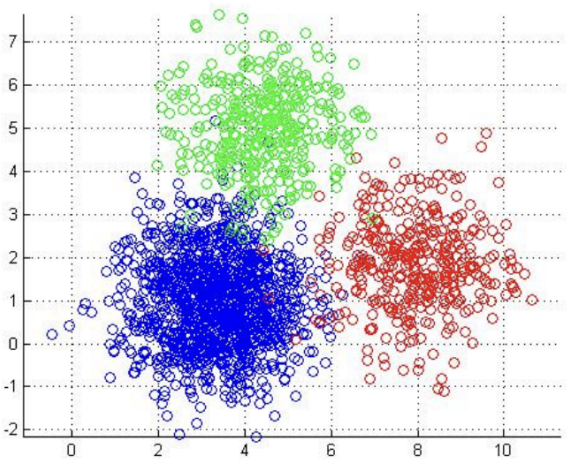
The learning changes based on if it is supervised and how the label set is specified. Y can be continuous or discrete. Moreover the training set is said to be supervised if the pairs are mad of (x_i, y_i) and unsupervised if there is no label (x_i)



Types of Learning

2) Unsupervised Learning (Clustering)

In unsupervised training there are **no labels**, this means that the **dataset is made only of instances** $(x_1, x_2, \ldots, x_m)$. The aim of our learner therefore becomes to **find a *structure* in the data** that we can exploit. The most used approach is called **clustering**, this approach is used in information retrieval, image processing, social analysis, ...



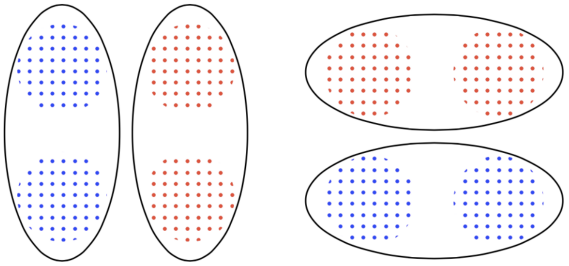
Example

Definition 1 (Clustering Informal Definition)**.**

Clustering is the task of grouping a set of objects such that *similar* objects end up in the same group and *dissimilar* objects are separated into different groups

Notice that this definition doesn't include a *real* definition of what similar means. The two approaches we will study will propose different solutions for different use cases, but will have to solve some common difficulties

Similarity is not *transitive*: that is, there si no ground truth for how to cluster
Example: we have points in $\mathbb{R}^2$



Both Clusters are justifiable

Definition 2 (Clustering)**.**

Given as **input** a set of elements $X = (x_1, \dots, x_m)$ and a **distance function** $d : X \times X \to \mathbb{R}_+$ that satisfies the following:

- Symmetry: $d(x, y) = d(y, x)\ \forall x, y \in X$
- Zero: $d(x, x) = 0\ \forall x \in X$
- Triangular Inequality: $d(x, y) \leq d(x, z) + d(z, y)$

The algorithm will **output** a partition of X into **clusters**, that is:

$$C = (C_1, \dots, C_k) \text{ with } \bigcup_i^k C_i = X,\ \forall i \neq j : C_i \cap C_j = \varnothing$$

Remark.

Sometimes instead of the distance function we use a similarity function $s : X \times X \to [0, 1]$ that satisfies the following:

- Symmetry: $s(x, y) = s(y, x)$
- Identity: $s(x, x) = 1$

Moreover we might have one more input parameter k that specifies how many clusters we want to have

2.1) Cost Minimization Algorithm: k-Means Clustering

As the name suggests this approach consists of defining a cost function over possible partitions of the objects and to find the partition of minimal cost. We turn the clustering problem in an optimization problem.

Assumptions:

- The data points $x \in X$ come from a larger space X' such that $X \subseteq X'$
- There exists the distance function $d(x, y)$ of $x, y \in X'$
  Moreover for simplicity we can assume $X' = \mathbb{R}^d$ and the euclidean distance
  $d(x, y) = ||x - y|| = \sqrt{\sum_1^d (x_i - y_i)^2}$

Inputs:
Data points $x_1, \dots, x_m \in \mathbb{R}^d$ and $k \in \mathbb{N}^+$

Goal:
find

- Partitions $C = (C_1, \dots, C_k)$ of $x_1, \dots, x_m$
- Centers $\mu_1, \dots, \mu_k \in \mathbb{R}^d$
  these must minimize one of these functions:
- **k-means obj function** (most common):

$$\min_{\mu_j \in \mathbb{X}' \forall j} \sum_{i=1}^{k} \sum_{x \in C_i} d(x, \mu_i)^2$$

this measures the squared distance of each point to the center of it's cluster

- **k-medoids objective function:** $\min_{u_j \in X} \sum \sum d(x, \mu_i)^2$ here the centers must be part of the dataset ($\mu_j \in X$ not $\in X'$)
- **k-median objective function:** $\min_{u_j \in X} \sum \sum d(x, \mu_i)$ no square of distance

Theorem 3 (Centroid)**.**

*Given a cluster C_i, we call **centroid** the center μ_i that minimizes $\sum_{x \in C_i} d(x, \mu_i)^2$, that is:*

$$\mu_i(C_i) = \arg\min_{\mu \in X'} \sum_{x \in C_i} d(x, \mu)^2 = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

Proof: (TODO)
Consider a generic point $y \in \mathbb{R}^d$ and the number of points in the cluster $n = |C_i|$. We have that:

$$d(x, y)^2 = ||x - y||^2 = ||x - \mu_i + \mu_i - y||^2 = (x - \mu_i)^2 + (y - \mu_i)^2 + 2(x - \mu_i)(-y + \mu_i)$$

Put this in the sum (sum with no indeces is from 1 to d and the summands are the i-th component)

$$\sum_{x \in C_i} d(x, y)^2 = \sum_{x \in C_i} \left(\sum_{t=1}^{d} (x^{(t)} - \mu_i^{(t)})^2 + \sum_{t=1}^{d} (y^{(t)} - \mu_i^{(t)})^2 + 2 \sum_{t=1}^{d} (x^{(t)} - \mu_i^{(t)})(-y^{(t)} + \mu_i^{(t)}) \right)$$

The first two terms are a distance squared, the third term is the definition of the *inner product*

$$\sum_{x \in C_i} d(x, y)^2 = n \cdot d(y, \mu_i)^2 + \sum_{x \in C_i} d(x, \mu_i)^2 + 2(\mu_i - y) \sum_{x \in C_i} (x - \mu_i)$$

By definition of μ_i we have that $\sum_{x \in C_i} (x - \mu_i) = -n\mu_i + \sum_{x \in C_i} x = \vec{0}$. Therefore

$$\sum_{x \in C_i} d(x, y)^2 = n \cdot d(y, \mu_i)^2 + \sum_{x \in C_i} d(x, \mu_i)^2 > \sum_{x \in C_i} d(x, \mu_i)^2$$

this shows that any point y that is not mu_i has a greater sum of distances from the points in the cluster

□

Conjecture (computationally unfeasible)**.**

Finding the (real) optimal solution for k-means clustering is computationally unfeasible.

To do so we would need to compute all combinations of the m points in k clusters. This sum would have $O(\frac{k^m}{k!})$, $\Omega(k^{m-k+1})$.

To solve the computationality problem we use the following algorithm:

Pseudocode 4 (Lloyd's Algorithm)**.**

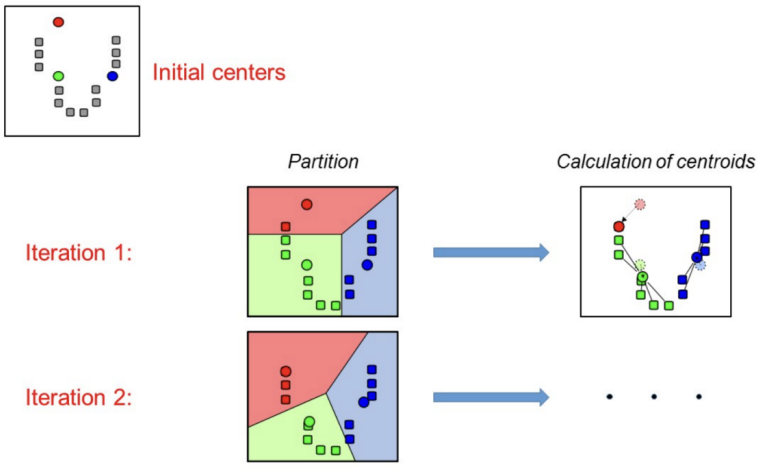
Lloyd's algorithm first defines random centers, then until convergence it does the following: Defines all clusters (based on distance), calculates new centers based on centroid thm.

Input: data points $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m\}$; $k \in \mathbb{N}^+$
Output: clustering $C = (C_1, C_2, \ldots, C_k)$ of $\mathcal{X}$; centers $\mu_1, \mu_2, \ldots, \mu_k$ with μ_i center for C_i, $1 \leq i \leq k$;

randomly choose $\mu_1^{(0)}, \ldots, \mu_k^{(0)}$;
for $t \leftarrow 0, 1, 2, \ldots$ **do** `/* until convergence */`
  for $i = 1, \ldots, k$: $C_i \leftarrow \{\mathbf{x} \in \mathcal{X} : i = \arg\min_j d(\mathbf{x}, \mu_j^{(t)})\}$;
  for $i = 1, \ldots, k$: $\mu_i^{(t+1)} \leftarrow \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$;
  if *convergence reached* **then**
    return $C = (C_1, \ldots, C_k)$ and $\mu_1^{(t+1)}, \mu_2^{(t+1)}, \ldots, \mu_k^{(t+1)}$

Pseudocode

visual representation



Example

There are many criteria of convergence, the following always does:

- The k-means objective at iteration t is not lower than that of iteration $t-1$

Another criteria is:

- For the convergence we define a value $\epsilon \in \mathbb{R}$. At the end of a cycle (t+1) we compare the total movement of the centers and if it is smaller than ϵ say it converges.

$$\sum_{1}^{k} d(\mu_i^{(t)}. \mu_i^{(t+1)}) \leq \epsilon$$

- Or we just look at the max

$$\max_{i\in[1,k]} d(\mu_i^{(t)}.\mu_i^{(t+1)}) \leq \epsilon$$

Theorem 5 (Lloyd's Complexity)**.**

The assignments of clusters is $O(kmd)$
The computation of centers is $O(md)$
Convergence after t *iterations* $\implies O(tkmd)$

Recent studies find that t has a lower bound in the worst case has $2^{\Omega\sqrt{m}}$
In practice we say it is much lass then m

Notice that t depends on the quality of the first random centers! A k-means++ algorithm is used, but we won't study it

2.2) Linkage-Based Clustering

Linkage Based Clustering is a straight forward approach where, starting from the trivial case where each datapoint is a single point cluster, is able to gradually decrease the number of clusters.

Idea of algorithm:

1. start from trivial clustering: each data point is a cluster
2. **until termination condition:** repeatedly merge the *closest* previous clusters

To apply this we need to identify 2 parameters:

1. distance between clusters
   there are 3 main distances we can define:

- **single linkage:**

$$D(A,B) = \min\{d(x,x') : x \in A, x' \in B\}$$

- **avg linkage:**

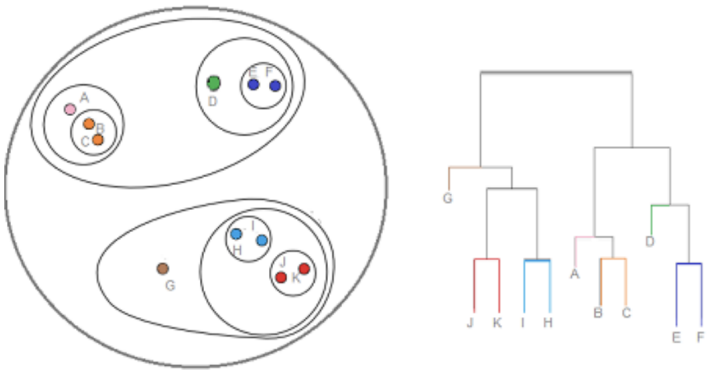
$$D(A,B) = \frac{1}{|A||B|}\sum_{x\in A, x'\in B} d(x,x')$$

- **max linkage:**

$$D(A,B) = \max\{d(x,x') : x \in A, x' \in B\}$$

2. termination condition

- there are k clusters
- min distance between clusters is $> r$ where r is a parameter
- all points in same cluster $\rightarrow$ output is a **dendrogram**



Dendrogram Example

Pseudocode 6 (Single Linkage Clustering Algorithm)**.**

The first loop creates the i clusters (one for each entry)

Then we create a PQ where we insert (nested loop) all distances between all points in form (distance [i,j], points [i,j])

The while loop corresponds to the second step:
Remove minimum distance from PQ, this is done until I find a pair of points [i,j] not belonging to same cluster
Then we merge: for all points of the clusters of i and j we all set them to the same cluster $\min\{C(x_j), C(x_i)\}$

Input: data points $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m\}$; termination condition;
Output: clustering of $\mathcal{X}$, with $C(\mathbf{x})$ being the cluster of $\mathbf{x}$;

for $i = 1, \ldots, m$ **do** $C(\mathbf{x}_i) \leftarrow i$;
$Q \leftarrow$ empty priority queue;
for $i = 1, \ldots, m-1$ **do**
  for $j = i+1, \ldots, m$ **do**
    Q.insert($d(\mathbf{x}_i, \mathbf{x}_j), (\mathbf{x}_i, \mathbf{x}_j)$);
while *termination condition* **do**
  $(d_{min}, (\mathbf{x}_i, \mathbf{x}_j)) \leftarrow Q$.removeMin();
  if $C(\mathbf{x}_i) \neq C(\mathbf{x}_j)$ **then**
    /* merge the clusters of $\mathbf{x}_i$ and $\mathbf{x}_j$ */
    set $C(\mathbf{x}_\ell) = \min\{C(\mathbf{x}_i), C(\mathbf{x}_j)\}$ for all $\mathbf{x}_\ell$ in the clusters of $\mathbf{x}_i$ and $\mathbf{x}_j$;
return clustering C;

Pseudocode

The **complexity** of the **first step** is the one of the PQ $\Theta(m^2)$
the **second step** has at most m^2 iterations. removeMin() has $O(\log m)$, the merge has $O(m)$ but with a better implementation $O(\log m), O(m^2 \log m)$
The final complexity is $O(m^2 \log m)$

2.3) How many clusters k do we need?

This is no easy task, here is the most used one:

- run the algorithm for various values of k and obtain many results
- use a **score** S to evaluate a score for different clusterings $S(C^{(k)})$ and choose the highest score

This is a common score based only on distance:

Theorem 7 (Silhouette)**.**

Given a clustering C of X and a point $x \in X$, let $C(x)$ be the point to which x is assigned to. Assume $|C_i| \geq 2$ (no single object cluster). Define

$$A(x) = \frac{\sum_{x' \in C(x)} d(x, x')}{|C(x)| - 1}$$

And given a cluster $C_i \neq C(x)$ let

$$d(x, C_i) = \frac{\sum_{x' \in C_i} d(x, x')}{|C_i|}$$

and

$$B(x) = \min_{C_i \neq C(x)} d(x, C_i)$$

*Then we can define the **silhouette** $s(x)$ of x:*

$$s(x) = \frac{B(x) - A(x)}{\max\{A(x), B(x)\}}$$

the silhouette is a measure to see if x is closer to it's cluster or to another
Finally the complete score

$$S(C) = \frac{\sum_{x \in X} s(x)}{|X|}$$

the higher the better
Moreover $s(x) \in (-1, 1)$

3) Empirical Risk Minimization

Let's define $D(E)$ as the probability of observing a point $x \in E \subset X$. Let E be defined by a function

$$\pi : X \to \{0,1\} : E = \{x \in X : \pi(x) = 1\}$$
$$\text{then: } P_{x \sim D}[\pi(x)] = D(E)$$

Definition 8 (Loss / Generalization error / Risk / True Error)**.**

The **error of the prediction rule** $h : X \to Y$ **is:**

$$L_{D,f}(h) = P_{x \sim D}[h(x] \neq f(x)) = D(\{x : h(x) \neq f(x)\})$$

The learner has h_S as the output. We want to fine h_S such that it minimizes $L_{D,f}(h)$ but by taking a closer look, it is clear that since both f and D are unknown also $L_{D,f}(h)$ can't be found.
Therefore we want to minimize the **training error**.

Definition 9 (Empirical / Training Error)**.**

We can define the **empirical error** as the error that can be calculated on the training set:

$$L_S(h) = \frac{|\{i : h(x_i) \neq y_i,\ 1 \leq i \leq m\}|}{m} = \frac{\text{wrong labels}}{\text{all samples}}$$

A new problem arises: **overfitting**

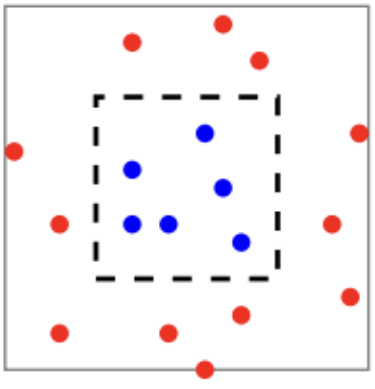
Our aim is to find a hypothesis h that minimizes $L_{D,f}(h)$, but since it can't be found we will be searching the hypothesis h that minimizes $L_S(h)$ while avoiding **overfitting.** We call this **Empirical Risk Minimization (ERM)**

Overfitting happens when the predictor fits the data too well. This following example will show a proof as to why overfitting, and therefore the *"raw"* ERM approach, is inaccurate:

In this example the small square has area 1, the bigger one area 2.
Consider the following hypothesis:

$$h(x) = \begin{cases} y_i & \text{if } x_i \text{ is in training set } S \\ 0 \end{cases}$$

With any training set we have the empirical error $L_S = 0$. Since this example was created by us we can also suppose to know D, f. By supposing a uniform distribution of points over the big square and that the labels have $y_i = 1$ if x_i is in the inner square, then we can also exactly compute the generalization error:



Randomized Samples

$$L_{D,f} = \frac{A_{small}}{A_{big}} = \frac{1}{2}$$

This error is equivalent to randomly choosing the label, we do not want it to be this high.

Probably Approximately Correct (PAC) Learning

A clever way to reduce overfitting is to apply an **inductive bias** to our hypothesis class and make it a **finite hypothesis class**.

Definition 10 (Finite Hypothesis Class)**.**

Our finite hypothesis class $\mathcal{H}$ is a set of hypothesis functions $h : X \to Y$ so that

$$|H| < \infty$$

Now our **ERM** is modified to find the smallest empirical risk in this finite set:

$$h_S = ERM_{\mathcal{H}} = \arg\min_{h \in \mathcal{H}} L_S(h)$$

This works under the following assumptions:
Assumptions

- **Realizability:** $\exists\, h^* \in H : L_{D,f}(h^*) = 0$
- **i.i.d:** the data in S are iid to D, that is $S \sim D^m$

The realizability assumption implies that if $L_D = 0 \implies L_S = 0$.
Moreover the i.i.d assumption guarantees that, since $D \sim S$, the training set can correctly direct the learning.

We can now define PAC learning by introducing these two following parameters

- **Confidence** δ**:** shows that h_s is good with probability $\geq 1 - \delta$
- **Accuracy** ε**:** we are satisfied with a h_s that has $L_{D,f}(h) \leq \varepsilon$
  $L_{D,f}$ is a random variable and therefore there will also be randomness in how h_S is chosen. The confidence is the measure of how good the training set is, δ is the probability of having a non representative sample. The accuracy is a measure on how good the output will be, since we cannot guarantee a perfect output.

Theorem 11 (PAC Learning)**.**

Let H be a finite hypothesis class. Let $\delta, \varepsilon \in (0,1)$ and $m \in \mathbb{N}$ the samples such that

$$m \geq \frac{\log(|H|/\delta)}{\varepsilon}$$

then for any f, D where the realizability assumption holds with probability $\geq 1 - \delta$ we have that for any hypothesis h_S it holds that

$$L_{D,f}(h_S) \leq \varepsilon$$

More accuracy $\varepsilon \to 0 \implies m \nearrow$

But the **realizability** assumption is too strong, it means that the label is fully determined by the instance, which is not always true.
Can we relax it? Yes, by supposing that D is the *joint* distribution over domain and labels ($X \times Y$). This tells us that there is no "true and perfect" solution, but we consider the data as a random pair over $X \times Y$. Therefore f is not needed anymore, let's change the definitions

$$\cancel{L_{D,f}(h) = P_{x \sim D}[h(x) \neq f(x)]} \qquad \to L_D(h) = P_{(x,y)\sim D}[h(x) \neq y]$$
$$\cancel{L_S(h) = \frac{|\{i : h(x_i) \neq y_i, 1 \leq i \leq m\}|}{m}} \qquad \to L_S(h) = \frac{|h(x_i) \neq y_i, 1 \leq i \leq m)|}{m}$$

We can generalize the loss function in the following way:

Definition 12 ((Generalized) Loss Function)**.**

Given any hypotheses set H and some domain Z, a loss function is any function $l : H \times Z \to \mathbb{R}_+$

Then we can rewrite the risk and the empirical risk as follows:

$$L_D(h) = \underset{z \sim D}{E}[l(h, z)]$$
$$L_S(h) = \frac{1}{m}\sum_{i=1}^{m} l(h, z_i)$$

Here are some commonly used loss functions:

$$\text{0-1 Loss:} \qquad l_{0-1}(h, (x, y)) = \begin{cases} 0 \text{ if } h(x) = y \\ 1 \end{cases}$$
$$\text{Squared Loss:} \qquad l_{\text{sq}}(h, (x, y)) = (h(x) - y)^2$$

Moreover, as we will see, $Z = X \times Y$ due to the relaxation of the realizability assumption.

4) Linear Models

Consider $X \in \mathbb{R}^d$, the **class of linear (affine) functions:**

$$L_d = \{h_{w,b} : w \in \mathbb{R}^d, b \in \mathbb{R}\} \text{ where } h_{w,b}(x) = \langle w, x \rangle + b = \left(\sum_{i=1}^{d} w_i x_i \right) + b$$

this means that each member of L_d is a linear function $x \to \langle w, x \rangle + b$ (with b the bias term).

The hypothesis class will have some minor changes:

- $h \in \mathcal{H}$ is now a function of type $h : \mathbb{R}^d \to Y$
- The hypothesis class itself is of type $\mathcal{H} : \phi \circ L_D$ ($\circ$ is the symbol for function composition: $\phi(L_D)$)

Moreover $\phi : \mathbb{R} \to Y$ depends on the problem itself:

- Binary Classification: $Y = \{-1, 1\} \to \phi(z) = \text{sign}(z)$
- Regression: $Y = \mathbb{R} \to \phi(z) = z$

Proposition 13 (Equivalent Notation)**.**

Given $x \in X \subseteq \mathbb{R}^d, w \in \mathbb{R}^d, b \in \mathbb{R}$ we define:

- $w' = (b, w_1, \dots, w_d) \in \mathbb{R}^{d+1}$
- $x' = (1, x_1, \dots, x_d) \in \mathbb{R}^{d+1}$

Then we can set

$$h_{w,b} = \langle w, x \rangle + b = \langle w', x' \rangle$$

4.1) Linear Classification

We talk of linear classification when:

- **Domain Set:** $X = \mathbb{R}^d$
- **Label Set:** $Y = \{-1, 1\}$
- **Loss:** 0-1 Loss: $l_{0-1}(h, (x, y)) = \begin{cases} 0 \text{ if } h(x) = y \\ 1 \end{cases}$
- **Training Set:** $S = ((x_1, y_1), \dots, (x_n, y_n))$
- **Hypothesis:** finite class of halfspaces: $\mathcal{H} = \text{sign } \circ L_d = \{x \to \text{sign}(h_{w,b}(x)) : h_{w,b} \in L_d\}$ that is, that the hypothesis class returns the sign of $\langle w, x \rangle + b$

As we know a good hypothesis is the one that **minimizes the training error (ERM)**.
The **working assumption** is that the data is **Linearly Separable**, that is

$$\exists w : y_i \langle w, x_i \rangle > 0$$

This confirms the realizability assumption for the training set.
The meaning is that the training data can be correctly separated in 2 spaces.

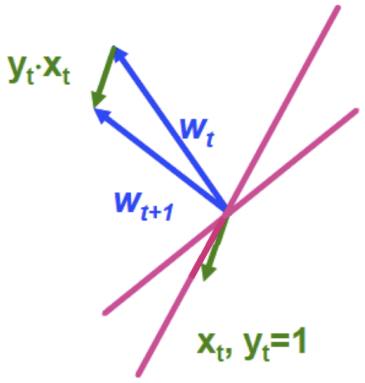
Pseudocode 14 (Perceptron Algorithm)**.**

In each loop we check if there is an instance (i) where the inner product is less than 0 this means that the realizability condition is not held, we must change w. It changes w by adding $y_i x_i$

Input: training set $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$
initialize $\mathbf{w}^{(1)} = (0, \ldots, 0)$;
for $t = 1, 2, \ldots$ **do**
  if $\exists i$ *s.t.* $y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle \leq 0$ **then** $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + y_i \mathbf{x}_i$;
  else return $\mathbf{w}^{(t)}$;

Algorithm

How does w change at each iteration?



Change in R2

We know that w is perpendicular to the division line, by adding a vector we change w and therefore the line.
this is due to

$$\begin{aligned} y_i \langle w^{(t+1)}, x_i \rangle &= y_i \langle w^{(t)} + x_i y_i, x_i \rangle \\ &= y_i \langle w^{(t)}, x_i \rangle + ||x_i||^2 \end{aligned}$$

Now with $y_i \langle w^{(t)}, x_i \rangle \leq 0$ we have that $y_i \langle w^{(t+1)}, x_i \rangle > y_i \langle w^{(t)}, x_i \rangle$

Once we assume that the data is separable (realizability assumption) we can derive this theorem:

Theorem 15 (Termination)**.**

Assume that the training set is linearly separable, let:

- $B = \min\{||w|| : y_i \langle w, x_i \rangle \geq 1 \; \forall i \in [1, m]\}$
- $R = \max_i ||x_i||$

Then the perceptron stops after at most $(RB)^2$ and $\forall i : y_i \langle w^t, x_i \rangle > 0$

Be careful, the perceptron gives one solution, but there are infinite ones and some might be better than others for non training set data.

If the data is **non separable** the perceptron has a variant *pocket algorithm* that runs for some times and stops with best solution found so far.

4.2) Linear Regression

We talk of linear classification when:

- **Domain Set:** $X = \mathbb{R}^d$
- **Label Set:** $Y = \mathbb{R}$
- **Loss:** Squared Loss: $l_{sq}(h,(x,y)) = (h(x) - y)^2$, from here the empirical loss called **Mean Squared Error**: $L_S(h) = \frac{1}{m}\sum_{0}^{m}(h_{x_i} - y_i)^2$
- **Training Set:** $S = ((x_1, y_1), \dots, (x_n, y_n))$
- **Hypothesis:** $\mathcal{H}_{reg} = L_d = \{x \to \langle w, x \rangle + b : w \in \mathbb{R}^d, b \in \mathbb{R}\}$ (finite affine functions set)

Our best ERM hypothesis is as always the argmin, we call this algorithm the **Least Squares Algorithm**:

$$\arg\min_w L_S(h_w) = \arg\min_w \frac{1}{m}\sum_{0}^{m}(h(x_i) - y_i)^2$$

An equivalent formulation is to find the w that minimizing the **Residual Sum of Squares (RSS)**. They are equivalent since:

$$w = \arg\min_w L_s(h_w) = \arg\min_w \frac{1}{m}\sum_{i=1}^{m}(h(x) - y)^2 = \arg\min_w \sum_{i=1}^{m}(\langle w, x_i \rangle - y_i)^2$$

Let's analyze this in matrix form:

$$X = \begin{bmatrix} \dots & x_1 & \dots \\ \dots & \dots & \dots \\ \dots & x_m & \dots \end{bmatrix},\ y = \begin{bmatrix} y_1 \\ \dots \\ y_m \end{bmatrix} \implies RSS = (y - Xw)^T(y - Xw)$$

we call X the **design matrix**. Our ERM hypothesis is the one that minimizes RSS.

Theorem 16 (Linear Regression Solution)**.**

Given a training set S and by building the $m \times 1$ matrix y and the $m \times (d+1)$ design matrix X the best ERM hypothesis is:

$$w = (X^T X)^{-1} X^T y$$

Proof:

1. First we show how RSS is obtained:
   Recall that $\hat{y}_i = \langle w, x_i \rangle = x_i^T w$ and now we can define the vector of predictions

   $$Xw = \begin{bmatrix} \hat{y}_1 \\ \dots \\ \hat{y}_m \end{bmatrix}$$

   The residuals is the difference between $\hat{y}_i - y_i$ and thus the vector of residuals is

   $$y - Xw = \begin{bmatrix} y_1 - \hat{y}_1 \\ \dots \\ y_m - \hat{y}_m \end{bmatrix}$$

and now clearly the square of residuals is $(y - Xw)^T(y - Xw) = \sum(y_i - \langle x_i, w\rangle)^2 = \sum(\langle w, x_i\rangle - y_i)^2 = RSS(w)$

2. Now let's prove the theorem
   The solution is the argmin of RSS. To find it we compute the gradient of RSS that is **convex** (global minimum) and compare it to $\vec{0}$.
   It is possible to show that

$$\nabla RSS(w) = \left(\frac{\partial RSS(w)}{\partial w_0}, \dots, \frac{\partial RSS(w)}{\partial w_d}\right) = -2X^T(y - Xw)$$

   This is due to:

- Expand RSS: $RSS = \sum(y_i - \langle w, x_i\rangle)^2 = \sum(y_i^2 + (\sum w_k x_i)^2 - 2y_i \sum w_k x_i)$
- Calculate the partial derivative:

$$\frac{\partial RSS(w)}{\partial w_i} = \sum(2\langle w, x_i\rangle \cdot x_i^{(1)} - 2y_i x_i^{(1)}) = -2\sum x_i^{(1)}(y_i - \langle w, x_i\rangle) = -2X_1^T(y - Xw)$$

  This is associated to the first feature of the instances in X^T
- Do it $\forall i$ and we obtain $RSS(w)$

By comparing it to $\vec{0}$ we obtain the form

$$-2X^T(y - Xw) = \vec{0} \rightarrow X^TXw - X^Ty = \vec{0} \rightarrow X^TXw = X^Ty$$
$$\overset{\text{if } X^TX \text{ invertible}}{\rightarrow} w = (X^TX)^{-1}X^Ty$$

□

The algorithm to compute $(X^TX)^{-1}X^Ty$ does the following 4 steps:

- Compute X^TX: product of $(d+1) \times m$ matrix and $m \times (d+1)$ matrix
- **Compute $(X^TX)^{-1}$: inversion of $(d+1) \times (d+1)$ matrix**
- Compute $(X^TX)^{-1}$: product between $(d+1) \times (d+1)$ matrix and $(d+1) \times m$ matrix
- Compute final value: compute product between $(d+1) \times m$ matrix and $m \times 1$ matrix
  The most computationally expensive operation is the inversion that has $O\left((d+1)^3\right)$.

What if X^TX is not invertible?
In this case we use the **generalized inverse**:
Let $A = X^TX$ then it's generalized inverse A^+ is given as $AA^+A = A$ and thus we have that $w = A^+X^Ty$

4.3) Polynomial Regression

Not all domains are linearly separable (ore are best separated linearly). To resolve this problem we **define the polynomial regression problem as a generalization of the linear regression.** The linear regression will be the same as the polynomial regression of degree 1.

We define the one dimensional polynomial as:

$$p(x) = w_0 + w_1x^1 + \dots + w_rx^r = \sum_{i=0}^{r} w_ix^i$$

Moreover we define the feature expansion vector:

$$x' = \begin{bmatrix} 1 \\ x \\ x^2 \\ \dots \\ x^r \end{bmatrix}$$

And the w vector:

$$w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \dots \\ w_r \end{bmatrix}$$

Then we can rewrite the polynomial as

$$p(x) = \langle w, x' \rangle$$

Finally the hypothesis class of the polynomial regression $\mathcal{H}^r_{poly} = \{x \to p(x)\}$ corresponds to the class of linear regression $\mathcal{H}_{reg}$ for x'.
If $X = \mathbb{R}^d$ then the feature expansion vector becomes

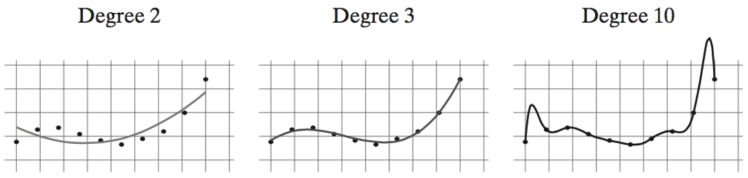
$$x' = [1, x_1, \dots x_1^r,\ x_2, \dots, x_2^r\ , \dots,\ x_d, \dots, x_d^r]^T$$

Therefore, once I fix a desired r the solution is the Least Squares Algorithm as seen in the linear regression.

How do I find the correct r? The answer is the next chapter, through **validation**.

5) Validation

How do I find the correct parameters for my algorithm?



Example

In this example the ER is null with degree 10 of the polynomial, but clearly the model is overfitted and the third degree seems to be the better choice.

We call **model selection** the step of choosing the right parameters and the right model for the desired learning problem

Validation is an additional step performed once a hypothesis is found. This step allows us to estimate the true (generalization) error of the resulting model.

Definition 17 (Validation Set / Hold Out Set)**.**

The validation set is a set of samples that aren't used to train the model.

$$V = \{(x_1, y_1), \dots, (x_{m_v}, y_{m_v})\}$$

Randomly sampling a validation set is equivalent to sampling the dataset and dividing it into two parts. One part is fed to the learner and the other is hold as a validation set. From here the name Hold Out Set.

Theorem 18 (Validation)**.**

Assume a loss function $l(h(x, y)) \in [0, 1]$*. Then,* $\forall \delta \in (0, 1)$ *with probability* $\geq 1 - \delta$ *we have that*

$$|L_V(h) - L_D(h)| \leq \sqrt{\frac{\log(2/\delta)}{2m_v}}$$

Where L_V *is the error on the validation set and* L_D *is the generalization error.* ***This result is valid for any algorithm and any distribution.***

Remark.

Recall that $L_V = \dfrac{1}{m_v} \sum_{1}^{m_v} l(h, (x_i, y_i))$

Moreover it is the case that

$$L_D > L_V \implies |L_V - L_D| = L_D - L_v \overset{\text{theorem}}{\implies} L_D \leq L_V + \sqrt{\frac{\log(2/\delta)}{2m_v}}$$

Validation can also be used for model selection: Suppose to have found r different hypothesis (recall the polynomial regression where we could have r degrees for the polynomial), then we have $\mathcal{H} = \bigcup_1^r \mathcal{H}_r$. The validation step will allow us to determine which hypothesis is the most performant one.

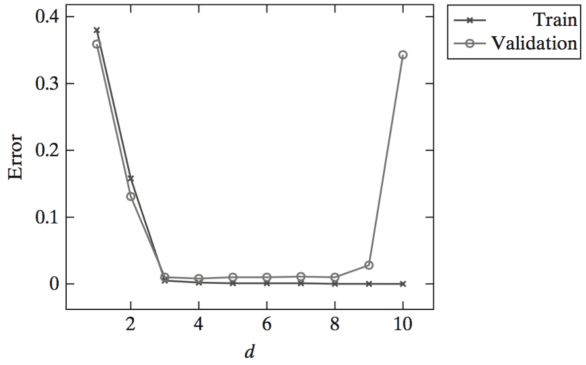
Theorem 19 (Validation For Model Selection)**.**

Let $\mathcal{H} = \cup \mathcal{H}_i$, then let h_i be the ERM hypothesis obtained on $\mathcal{H}_i$. Assume a loss function $l(h(x, y)) \in [0, 1]$. Then with probability $\geq 1 - \delta$ we have

$$\forall h \in \{h_i, \dots, h_r\} : |L_D(h) - L_V(h)| \leq \sqrt{\frac{\log(2r/\delta)}{2m_v}}$$

This theorem tells us that, as long as we use a small number of hypotheses, the error on the training set is not too different from the real error.

Now, by plotting the training and validation error in a graph called **model selection curve** it is possible to see when we get into overfitting



Model-Selection Curve

Here we see that **the training error is monotonically decreasing**, on the other hand the validation error will first decrease but then increase again.

In more advanced models it is possible to have more parameters in $\mathbb{R}$. Validation is still feasible although a bit more complicated:

1. Start with a rough grid of values
2. Plot the model-selection curve
3. Based on the curve zoom into the correct regime
4. Restart from 1) with the updated grid

The validation set should be a "fresh" dataset. in reality we have only one dataset that gets split in two parts: training and validation sets.

Train Validation Test Split: k-Fold Cross Validation

Let's split the data in three parts:

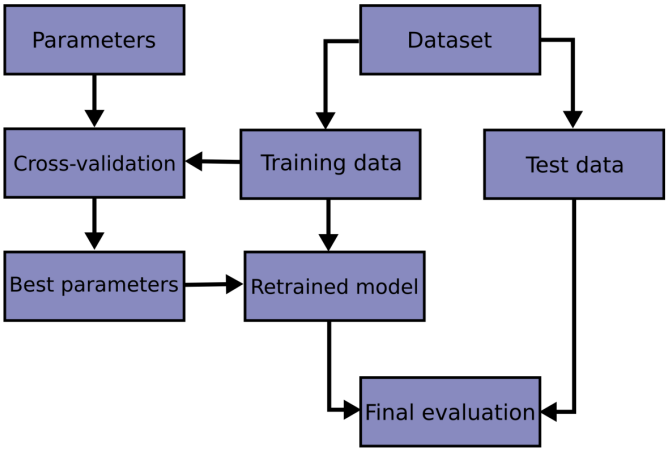
- training set: learn h_i from each $\mathcal{H}$
- validation set: choose $h \in \mathcal{H} = \bigcup H_i$ using Theorem 19 (Validation For Model Selection)
- test set: estimate $L_D(h)$ using Theorem 18 (Validation). This part of the data is not involved in the choice of h

For each $L_D(h)$ we must use a different test set in order to avoid biased results

When data is not plentiful we cannot use a fresh new validation set, therefore we use **cross validation**

1. Partition the training set into k folds of size m/k where m is the number of examples we have
2. for each fold: train on union of the other folds and then estimate the error on this fold
3. the estimated true error is the average of the estimated errors

This is used to find the best model (parameters) and then the model is retrained with this parameters. Finally the obtained model will use the test dataset to give the final estimation



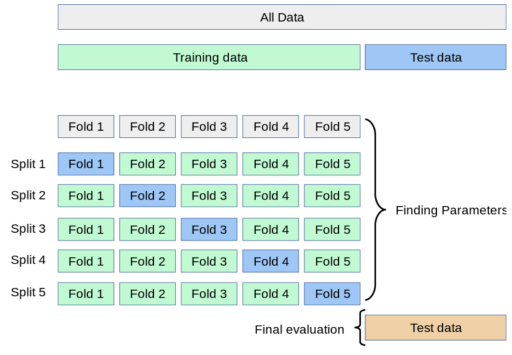
Diagram

Pseudocode 20 (k-Fold Cross Validation For Model Selection)**.**

input:
training set $S = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$
set of parameter values Θ
learning algorithm A
integer k
partition S into $S_1, S_2, \ldots, S_k$
foreach $\theta \in \Theta$
for $i = 1 \ldots k$
$h_{i,\theta} = A(S \setminus S_i; \theta)$
$\text{error}(\theta) = \frac{1}{k} \sum_{i=1}^{k} L_{S_i}(h_{i,\theta})$
output
$\theta^\star = \text{argmin}_\theta \, [\text{error}(\theta)]$
$h_{\theta^\star} = A(S; \theta^\star)$

Pseudocode

The particular case of $k = m$ (m is the number of examples) we have the case of **Leave-One-Out**:



Diagram

6) Basics from Statistics (Confidence Intervals, MLE, Hypothesis Testing)

This chapter will introduce to statistics. The study fixes a value μ obtained from a distribution and then to finding some useful data related to D, μ.

6.1) Confidence Set

Definition 21 (Confidence Set)**.**

We say that $I(x)$ is a confidence interval (for parameter θ) of level $1-\alpha$, if:

$$P[\theta \in I(x)] > 1-\alpha$$

This means that once we fix a value θ we find an interval that contains θ with high $(1-\alpha)$ probability. Recall that θ is fixed but $I(x)$ is given from a r.v. and thus is "randomly generated".
Frequentist point of view of $I(x)$: out of N independent Draws of $x \sim p_x(x;\theta)$ we expect that $(1-\alpha)100\%$ of times the interval $I(x)$ contains the true (but unknown) θ

Ideally we would like to let the set $I(x)$ be:

- small $\rightarrow$ high precision
- containing θ with high precision $\rightarrow$ high confidence

6.2) Maximum Likelihood Estimation

The Maximum Likelihood Estimation (MLE) is the statistical approach for finding the parameters that maximize the joint probability of a given dataset assuming a specific parametric probability function

MLE is the $\arg\max$ of the log likelihood:

- Given a training set S and the likelihood to extract a said value given some parameters $P[S|\theta]$, the **log likelihood** is $L(S;\theta) = \log(P[S|\theta])$
- The MLE is then $\hat{\theta} = \arg\max_\theta L(S;\theta)$

With a Gaussian random variable $X_i \sim \mathcal{N}(\theta, \sigma^2)$ we have

$$L(x,\theta) = -\frac{m}{2}\log(2\pi\theta^2) - \frac{1}{2}\sum_{i=1}^{m}\frac{(x_i-\theta)^2}{\sigma^2}$$

By computing the derivative and comparing it to 0 we can find the MLE as the simple mean:

$$\hat{\theta} = \arg\max L(x,\theta) = \frac{1}{m}\sum_{i=1}^{m} x_i \sim \mathcal{N}\left(\theta, \frac{\sigma^2}{m}\right)$$

This sample is also normal $\hat{\theta} \sim \mathcal{N}(\theta, \sigma^2)$ and thus for a confidence $1-\alpha$ the smallest confidence set for θ is symmetric and centered around the mean:

$$I(x) = [\hat{\theta} - z_{\alpha/2}\frac{\sigma}{\sqrt{m}}, \hat{\theta} + z_{\alpha/2}\frac{\sigma}{\sqrt{m}}]$$

where $P[\varepsilon \leq z_{\alpha/2}] = 1 - \frac{\alpha}{2}$ and $\varepsilon \sim \mathcal{N}(0,1)$. That is $z_{\alpha/2}$ is the $1-\alpha/2$ percentile of the zero mean unit variance Normal distribution $\mathcal{N}(0,1)$

6.3) (Statistical) Hypothesis Testing

Given a model we are also interested in verifying (actually falsifying, i.e. disprove) assumptions on the parameter θ .

The simplest instance would be to test the hypothesis $\theta = \theta_0$. That is: we find a region $\mathcal{R}(H_0)$ such that, given a sample $x \sim p_x(x;\theta)$, if $x \in \mathcal{R}(H_0)$ we reject H_0.

We have two types of errors:

- H_0 is true but $x \in R(H_0) \rightarrow$ reject H_0 (type I) $P = \alpha$
- H_0 is false but $x \notin R(H_0) \rightarrow$ accept H_0 (type II) $P = \beta$

With a gaussian r.v. of type $\mathcal{N}(0,1)$ the zero hypothesis is $\theta = \theta_0 = 0$ and here we find the biggest region $\mathcal{R}(H_0)$ in the form of:

$$\mathcal{R}(H_0) = \left\{ (x_1, \dots, x_m) : \left| \frac{1}{m} \sum_{i=1}^{0} x_i - \theta_0 \right| > z_{\alpha/2} \frac{1}{\sqrt{m}} \right\}$$

In the next chapter the meaning and usefulness of hypothesis testing will be made clear.

7) Significance Testing for Linear Regression Coefficients

Assumption: data is generated from a linear model + a noise of type $\phi \sim \mathcal{N}(0, \sigma^2)$ with σ known.
Model:

$$Y = w_0 + \sum_{i=1}^{d} w_i X_i + \phi$$

Least squares is used to estimate $\hat{w}_i$ of w_i. We will therefore just do an estimate. These will be distributed

$$\hat{w}_i \sim \mathcal{N}(w_i, v_i \sigma^2)$$

where v_i is the i-th diagonal element of $(X^T X)^{-1}$

In particular we are interested in the **null hypothesis** of $w_i = 0$ since if σ^2 is known, then we define the normalized variable as:

$$z_i = \frac{\hat{w}_i}{\sigma\sqrt{v_i}} \sim \mathcal{N}(0, 1)$$

and therefore the $1 - \alpha$ confidence interval for w_i is then

$$I(w_i = 0) = \left(\hat{w}_i - z_{\alpha/2}\sigma\sqrt{v}_i, \hat{w}_i + z_{\alpha/2}\sigma\sqrt{v}_i\right)$$

Change of assumption: σ^2 is unknown
We can still find MLE of σ^2 as

$$\hat{\sigma}^2 = \frac{1}{m-d-1}\sum_{1}^{m}(y_j - \hat{y}_j)^2 = \frac{1}{m-d-1} RSS(\hat{w})$$

where $\hat{y}_j = \hat{w}_0 + \sum_1^d \hat{w}_i x_j^{(i)}$ with $x_j^{(i)}$ the i-th feature of the j-th sample and has the following corresponding normalized variable:

$$z_i = \frac{\hat{w}_i}{\hat{\sigma}\sqrt{v_i}}$$

Remark: here we used the estimate $\hat{\sigma}$, not σ.
Finally, the $1 - \alpha$ confidence interval for w_i is then:

$$I(w_i) = \left(\hat{w}_i - t_{\alpha/2}^{(m-d-1)}\sqrt{v_i}\hat{\sigma}, \hat{w}_i + t_{\alpha/2}^{(m-d-1)}\sqrt{v_i}\hat{\sigma}\right)$$

Notice how we have $m - d - 1$ degrees of freedom.
In this interval we have that $t_{\alpha/2}^{(m-d-1)}$ is the $1 - \alpha/2$ percentile of the Students t distribution. This holds since z_i follows a student's t distribution with $m - d - 1$ degrees of freedom: $z_i \sim t_{m-d-1}$

Why are we doing this?
Significance testing is a statistical way to determine the importance of the parameters in our regression models. In both cases we aim to test the null hypothesis $H_0 : w_i = 0$. If the confidence interval contains the null hypothesis ($0 \in I(w_i)$), then we have no evidence to reject H_0. Moreover if $0 \notin I(w_i)$ then we can reject H_0 and the feature is relevant in our model.

8) Gaussian Mixture Model (GMM) and Expectation-Maximization (EM)

A **Gaussian Mixture Model (GMM)** is a probabilistic model that assumes your data is generated from a mixture of several Gaussian (normal) distributions, each with its own mean and variance.

Assumption: the data $x_1, \dots, x_m$ is generated from k Gaussian distributions with different parameters:

- mean $\mu_i \in \mathbb{R}^d$
- covariance matrix $\Sigma_i \in \mathbb{R}^{d \times d}$

Therefore a point x generated from the i-th Gaussian is:

$$Pr(x|\mu_i, \Sigma_i) = \frac{1}{(\sqrt{2\pi})^d \sqrt{\det(\Sigma_i)}} e^{-\frac{(x-\mu_i)^T \Sigma_i^{-1} (x-\mu_i)}{2}} := p_i(x)$$

and finally, the **density function** of x that comes from a GMM over k gaussians $C_1, \dots C_k$ is:

$$p(x) = \sum_{i=1}^{k} p_i(x) Pr(C_i) = \sum_{i=1}^{k} Pr(x|\mu_i, \Sigma_i) Pr(C_i)$$

Where $Pr(C_i)$ is the a priori probability that x comes from the i-th gaussian. These probabilities must satisfy $\sum Pr(C_i) = 0$.

Our model will have the following parameters: $\sigma = \{\mu_1, \Sigma_1, Pr(C_1), \dots, \mu_k, \Sigma_k, Pr(C_k)\}$

To obtain a **clustering** you just assign x_j to the C_i that maximizes $Pr(C_i|x_j)$ that can be computed by Bayes Theorem: $\Pr(C_i \mid x_j) = \frac{\Pr(x_j|C_i) \cdot \Pr(C_i)}{\Pr(x_j)}$

Given a dataset $D = (x_1, \dots, x_m)$ of iid samples from a GMM we can find the k-clustering of these samples using the **maximum likelihood approach**:
The likelihood of D given θ is

$$Pr(D|\theta) = \prod p(x_j)$$

And the Maximum Likelihood Estimation (MLE) aims to find θ^* that maximizes the likelihood of the data:

$$\theta^* = \arg\max_{\theta} Pr(D|\theta) = \arg\max_{\theta} \sum p(x_j)$$

Instead of using the classic likelihood we use the equivalent notation of the log-likelihood. For the GMM we end up with:

$$\ln Pr(D|\theta) = \sum_{j=1}^{m} \ln \left(\sum_{i=1}^{k} Pr(x_j|\mu_i \Sigma_i) Pr(C_i) \right)$$

But finding the argmax of this expression is really hard to solve!

Expectation-Maximization (EM)

The algorithm will find a **local** minimum as the function is not convex. The working principle of the algorithm is:

Theorem 22.

The log-likelihood improves at each iteration

Corollary.

The algorithm converges to a local optimum

The algorithm works in the following way.
Initialization:

- μ_i take value uniformly at random
- $\Sigma_i = I_{ID}$
- $Pr(C_i) = 1/k$

Repeat the next 2 steps until convergence
Expectation:
What is the current estimate that x_j comes from C_i for all i and j

$$w_{ij} := Pr(C_i|x_j) = \frac{p_i(x_j)Pr(C_i)}{\sum_1^k} p_h(x_j)Pr(C_h)$$

Where w_{ij} is the contribution of x_j to C_i
Maximization
For each $i = 1, \dots, k$ we update the parameters

$$\mu_i = \frac{\sum_{j=1}^{m} w_{ij}x_j}{\sum_1^m w_{ij}}$$

$$\Sigma_i = \frac{\sum_1^m w_{ij}(x_j - \mu_i)(x_j - \mu_i)^T}{\sum_1^m w_{ij}}$$

$$Pr(C_i) = \frac{\sum_1^m w_{ij}}{m}$$

Pseudocode 23.

Input: data points $D = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m)$, $k \in \mathbb{N}^+$, accuracy parameter ε
Output: w_{ij}, and estimates of μ_i, Σ_i, and $\Pr(C_i)$ for $i = 1, \ldots, k$, $j = 1, \ldots, m$

$t \leftarrow 0$;
`/* Initialization`
randomly initialize $\mu_1^{(t)}, \ldots, \mu_k^{(t)}$;
$\Sigma_i^{(t)} \leftarrow I, \forall i = 1, \ldots, k$;
$\Pr^{(t)}(C_i) \leftarrow 1/k, \forall i = 1, \ldots, k$;
repeat
  `/* Expectation Step`
  for $i = 1, \ldots, k$ *and* $j = 1, \ldots, m$ **do**
    $w_{ij} \leftarrow \frac{\Pr(\mathbf{x}_j | \mu_i^{(t)}, \Sigma_i^{(t)}) \Pr^{(t)}(C_i)}{\sum_{h=1}^{k} \Pr(\mathbf{x}_j | \mu_h^{(t)}, \Sigma_h^{(t)}) \Pr^{(t)}(C_h)}$
  `/* Maximization Step`
  $t \leftarrow t + 1$;
  for $i = 1, \ldots, k$ **do**
    $\mu_i^{(t)} \leftarrow \frac{\sum_{j=1}^{m} w_{ij} \mathbf{x}_j}{\sum_{j=1}^{m} w_{ij}}$;
    $\Sigma_i^{(t)} \leftarrow \frac{\sum_{j=1}^{m} w_{ij} (\mathbf{x}_j - \mu_i^{(t)})(\mathbf{x}_j - \mu_i^{(t)})^T}{\sum_{j=1}^{m} w_{ij}}$;
    $\Pr^{(t)}(C_i) \leftarrow \frac{\sum_{j=1}^{m} w_{ij}}{m}$;
until $\sum_{i=1}^{k} ||\mu_i^{(t)} - \mu_i^{(t-1)}||^2 \leq \varepsilon$;
return $w_{ij}, \mu_i^{(t)}, \Sigma_i^{(t)}$, and $\Pr^{(t)}(C_i)$ for $i = 1, \ldots, k$, $j = 1, \ldots, m$;

Pseudocode